Project title : “Script-Controlled ACL - Restrict Record Access Based on Field Value.”

Phase 1 : Brainstorming & Ideation Phase

1. Project Introduction and Overview

This project focuses on implementing a **script-controlled Access Control List (ACL)** within the **ServiceNow** platform to restrict access to records based on specific business conditions. The project is centered on the **Institution Details** (`u_institution_details`) table and demonstrates how record-level security can be applied when users attempt to access sensitive student information.

The primary objective is to ensure that record access is **not granted uniformly** across all platform users. Instead, the architecture leverages a combination of custom roles, field-level conditions, and advanced server-side script logic:

- **Target Security:** `EEE` branch records are protected so that only configured users can access permitted records.
- **Administrative Override:** System administrators retain full access across all records regardless of field conditions.
- **Multi-Operation Scope:** Extends access control beyond simple viewing by planning separate ACLs for **Create**, **Write**, and **Delete** operations through distinct custom roles.

2. In-Depth Problem Statement

In an enterprise environment storing institutional and student data, allowing unrestricted read or write permissions introduces significant security and compliance risks.

Key Challenges Addressed

1. **Unrestricted Record Exposure:**

- Fields such as **Student Roll Number**, **Student Name**, **Faculty Name**, **Email**, **Phone Number**, and **Description** require granular visibility controls.
- The platform must dynamically differentiate between records belonging to different academic branches (**ECE**, **EEE**, and **CSE**). Non-authorized users must be strictly prevented from viewing protected records.

2. **Monolithic Permission Structures:**
   - Standard access controls often group operational permissions together. Viewing, creating, updating, and deleting records are distinct operations that require independent governance.
   - Without operation-specific ACLs, security governance becomes rigid and difficult to manage.

3. Proposed Engineering Solution

The solution builds a robust ServiceNow security architecture around the `u_institution_details` table, utilizing custom roles, field conditions, and script logic.

Data & Role Matrix

Operation	Custom Role	ACL Type	Execution Logic / Condition
Read	`bb1`	Record / Read	Data Condition: `Branch IS EEE` + Advanced Server-Side Script
Create	`bb2`	Record / Create	Enforces creation permissions via role `bb2`
Write	`bb3`	Record / Write	Enforces modification permissions via role `bb3`
Delete	`bb4`	Record / Delete	Enforces deletion permissions via role `bb4`

Script-Controlled Security Logic

The **Read ACL** forms the primary security boundary. The server-side script checks:

1. Whether the active user possesses the `admin` role (grants full access).
2. Whether the active user possesses the `bb1` role alongside meeting the `Branch == EEE` condition.
3. Denies access dynamically for all non-matching requests.

4. Target Audience & Key Stakeholders

- **ServiceNow Administrators & Developers:** Gain a clear, practical model for combining script logic, platform roles, and table conditions to enforce strict record security.
- **Platform End Users:** Benefit from a structured, safe data environment where field visibility and editing capabilities match specific job responsibilities.
- **System Auditors & Security Teams:** Receive a clear framework demonstrating compliance, role separation, and least-privilege access implementation.

🔗 Important Technical References & Resources

- 📘 ServiceNow Official Documentation: Access Control Lists (ACLs)
- 🛠️ ServiceNow Developer Program & Personal Developer Instances (PDI)
- ⚡ ServiceNow Community: Scripting in Access Control Rules

Phase 2 : Requirement Analysis Phase

1. Project Requirement Overview

The project requires a **ServiceNow Personal Developer Instance (PDI)** with administrator-level privileges (`admin` role) to configure users, roles, tables, and Access Control Lists (ACLs).

Core Environment Configurations

- **Test User:** Create a test user named `EEE User` via `User Administration → Users`.
- **Custom Security Roles:** Create four distinct custom roles via `User Administration → Roles`:
  - `bb1` — Designated for **Read / View** access.
  - `bb2` — Designated for **Create** access.
  - `bb3` — Designated for **Write / Edit** access.
  - `bb4` — Designated for **Delete** access.

2. Data & Table Requirements

A custom application table must be created to store student information securely.

Table Schema: `u_institution_details`

Field Name	System Name	Data Type	Configuration / Details
Student Roll Number	`u_student_roll_number`	**Auto Number**	Unique primary identifier for records
Student Name	`u_student_name`	**Reference**	Points to the `sys_user` table
Faculty Name	`u_faculty_name`	**Reference**	Points to the `sys_user` table
Branch	`u_branch`	**Choice**	Dropdown choices: `ECE`, `EEE`, `CSE`
Email	`u_email`	**String**	Contact email address
Phone Number	`u_phone_number`	**String**	Contact phone number
Description	`u_description`	**String (Multi-Line)**	Additional remarks and notes

📌 **Data Setup:** Insert multiple test records spanning across **ECE**, **EEE**, and **CSE** branches to effectively validate field-level and record-level script execution.

3. Functional & Security Requirements

The core requirement is to enforce fine-grained record security on the `u_institution_details` table using both condition builders and advanced server-side scripting.

ACL Matrix Overview

ACL Operation	Target Table	Associated Role	Condition / Execution Type
Read	`u_institution_details`	`bb1`	**Data Condition:** `Branch IS EEE` + **Advanced Script**
Create	`u_institution_details`	`bb2`	Role-Based Access Control
Write	`u_institution_details`	`bb3`	Role-Based Access Control
Delete	`u_institution_details`	`bb4`	Role-Based Access Control

Advanced Read ACL Script Logic

To enforce advanced record-level security, toggle the **Advanced** checkbox on the Read ACL form and apply the following server-side logic:

JavaScript
```
// Advanced Script for u_institution_details Read ACL
(function executeRule(current, previous /*null when async*/) {
```

```
    // 1. System Administrators retain unrestricted access
    if (gs.hasRole('admin')) {
        answer = true;
        return;
    }

    // 2. Permit user if they hold role 'bb1' AND record branch is 'EEE'
    if (gs.hasRole('bb1') && current.u_branch == 'EEE') {
        answer = true;
    } else {
        // 3. Restrict access for all other conditions
        answer = false;
    }

})(current, previous);
```

4. Verification & Testing Requirements

To verify the project implementation, perform system impersonation tests across three distinct user profiles:

[System Administrator] ──► Full access to all branches (ECE, EEE, CSE)
[EEE User (Role: bb1)] ──► Access restricted ONLY to 'EEE' branch records
[Unauthorized User] ──► Zero record visibility (Security constraint message)

Verification Test Cases

1. **`EEE User` Test:** Impersonate `EEE User` (assigned role `bb1`). Navigate to **`Institution Details → All Records`**. Verify that **only records where `Branch == EEE`** are displayed.
2. **Unauthorized User Test:** Impersonate a standard user without `bb1` or `admin` roles. Confirm that zero records are returned and platform security warnings appear.
3. **Admin User Test:** Confirm that administrators continue to see all records across `ECE`, `EEE`, and `CSE` branches without restriction.
4. **CRUD Operation Progression:** Incrementally assign `bb2`, `bb3`, and `bb4` roles to test that **Create**, **Write**, and **Delete** buttons dynamically appear and function as expected.

🔗 Useful Development Links & Documentation

- 🌐 ServiceNow Developer Site: Personal Developer Instances (PDIs)
- 📖 ServiceNow Product Documentation: Table Creation & Data Types
- 🔐 ServiceNow Docs: Access Control List (ACL) Rules
- 💡 ServiceNow Community: Advanced ACL Scripting Patterns

Phase 3 : Project Design Phase

1. Overall System Design

The system is designed as a record-level security solution within ServiceNow. The protected data is stored in the `u_institution_details` table, and access to that data is controlled through Access Control Lists (ACLs).

The architectural design contains several connected components:

- **EEE User & Custom Roles (`bb1`, `bb2`, `bb3`, `bb4`)**
- **Institution Details Target Table (`u_institution_details`)**
- **Student Records Data Structure**
- **Operation-Specific Access Control Rules (Read, Create, Write, Delete ACLs)**

The system separates access operations so that each action can be independently controlled, establishing a granular permission model rather than a single broad access grant.

2. Database and Field Design Schema

The `u_institution_details` table is designed to store and manage student-related information securely.

- **Student Roll Number:** Generated through System Auto-Number mechanism.
- **Student Name & Faculty Name:** Configured as User Reference Fields (`sys_user`).
- **Branch:** Choice Field containing values: `ECE`, `EEE`, and `CSE`.

- **Email & Phone Number:** Standard String Fields.
- **Description:** Multi-Line String Field.

Multiple records containing different branch values are maintained to validate security boundaries during testing.

3. Role and Security (ACL) Design

The security model isolates operations using four primary custom roles:

Operation	Target ACL	Required Role	Additional Logic / Data Condition
Read	`u_institution_details.read`	`bb1`	Branch is `EEE` + Advanced Script
Create	`u_institution_details.create`	`bb2`	None
Write	`u_institution_details.write`	`bb3`	None
Delete	`u_institution_details.delete`	`bb4`	None

The Read ACL enforces both a field condition (`Branch = EEE`) and advanced script logic to restrict unauthorized record access.

4. Script & Design Verification

The advanced Read ACL script executes three distinct evaluation steps:

1. **Admin Override Check:** Evaluates if the user possesses the `admin` role and grants immediate access.
2. **Role Verification:** Checks if the user holds the custom `bb1` role.
3. **Restricted Access Fallback:** Returns `false` for any user who fails to meet the role and branch requirements.

System Configuration & Execution Proof

> 📷 **[INSERT YOUR PROJECT SCREENSHOT HERE]**
>
> *(Recommendation: Read ACL form screenshot - Script section, `bb1` role, and `Branch is EEE` condition theriyura maathiri oru photo pottukonga)*

Phase 4 : Project Planning Phase

1. Project Implementation Plan

- The project is planned as a sequence of milestones beginning with user and role creation, followed by table and record creation, ACL configuration, verification, documentation, and demonstration.
- The first implementation activity is to access the ServiceNow instance with administrator privileges. The EEE User is then created, followed by the creation of **bb1**, **bb2**, **bb3**, and **bb4** roles.
- After the identity and role foundation is ready, the **Institution Details** table and its fields are created. Records with **ECE**, **EEE**, and **CSE** branch values are then inserted for testing.

2. ACL Development Schedule

- The **Read ACL** is planned as the main security component because it demonstrates the field-based access requirement. The ACL is created with the read operation, **bb1** role requirement, **Branch is EEE** data condition, and advanced script.
- After Read access is configured, the **Create ACL** is created using **bb2**. The **Write ACL** follows using **bb3**, and the **Delete ACL** is created using **bb4**.
- This sequence allows the project to demonstrate the progression from basic viewing permission to record creation, modification, and deletion.

3. Testing and Verification Plan

- The testing plan begins with impersonating the **EEE User** and opening the **Student Records list**. The expected behavior is checked against the configured Read ACL.
- A user without the required role is then considered for verification to confirm that unauthorized access is denied. The **Administrator** is also tested to verify the documented full-access behavior.
- **Create**, **Write**, and **Delete** operations are tested after the corresponding roles are configured. The **New button**, **editing capability**, and **delete capability** are checked as part of the demonstration.

4. Project Deliverables and Timeline Structure

- The planned project deliverables include the configured ServiceNow environment, screenshots or evidence of user and role creation, table and field configuration, ACL configuration, testing results, documentation, and final demonstration material.
- The GitHub submission is organized into eight phases:
    1. Brainstorming & Ideation
    2. Requirement Analysis
    3. Project Design
    4. Project Planning
    5. Project Development
    6. Project Testing
    7. Project Documentation
    8. Project Demonstration
- Maintaining these phases separately provides a clear record of how the project progresses from the original idea to the final security demonstration.

Phase 5 : Project Development Phase

1. User and Role Development

- Development begins by logging in to the ServiceNow instance with administrator access. A test user named **EEE User** is created through **`User Administration → Users`**.
- The required custom roles are created through **`User Administration → Roles`**. The roles are named **`bb1`**, **`bb2`**, **`bb3`**, and **`bb4`**, and the **EEE User** is assigned the roles required by the project.
- These roles become the permission components used by the four ACLs. The development therefore establishes the identity and authorization structure before the protected table is configured.

2. Institution Details Table Development

- The custom **Institution Details** table is created through **`System Definition → Tables`**. The label is **Institution Details** and the table name is **`u_institution_details`**. The table is configured without extending another table.
- The required fields are added after the table is created:
  - **Student Roll Number:** Auto Number field
  - **Student Name & Faculty Name:** User Reference fields (`sys_user`)
  - **Branch:** Choice field containing **ECE**, **EEE**, and **CSE**
  - **Email & Phone Number:** String fields
  - **Description:** Multi String field
- The table configuration is saved after all fields are properly defined.

3. Record and Read ACL Development

- Multiple records are created in the table using different **Branch** values. This provides **ECE**, **EEE**, and **CSE** records for ACL testing.
- The **Read ACL** is created under **`System Security → Access Control`** after elevating privileges with the **`security_admin`** role. It is configured as a record-level ACL for the read operation on **`u_institution_details`**.
- **`bb1`** is added under **Requires role**, **`Branch is EEE`** is configured as the data condition, and **Advanced** is enabled.
- The script is added so that administrators are allowed, the **`bb1`** condition is handled, and other users are denied access.

Development Configuration & ACL Script Proof

📷 **[INSERT YOUR PROJECT DEVELOPMENT SCREENSHOT HERE]**

(Recommendation: Add a screenshot of the `u_institution_details` Read ACL configuration page showing the `bb1` role, `Branch is EEE` condition, and Advanced Script block)

4. Create, Write, and Delete ACL Development

- The **Create ACL** is created for the record create operation and requires the **bb2** role. No additional data condition is added.
- The **Write ACL** is created for the record write operation and requires the **bb3** role. It is saved without an additional data condition.
- The **Delete ACL** is created for the record delete operation and requires the **bb4** role.
- After all ACLs are saved, the project development phase is complete and ready for verification.

Phase 6 : Project Testing Phase

1. Testing Strategy

The testing phase verifies whether the ACL configuration behaves according to the project's defined requirements. Testing is performed using different user roles and record operations.

The main Read ACL test uses the EEE User and the Student Records list. Additional tests examine users without the required role and administrators.

Separate tests are then performed for Create, Write, and Delete operations after the corresponding roles have been configured.

- **Testing Approach & Environment:** Tests are executed using ServiceNow's native **Impersonation** feature to dynamically validate role assignments without altering security credentials.

- **Scope of Verification:** Covers both **Table-level ACLs** (restricting access to the `u_institution_details` table as a whole) and **Field-level ACLs** (controlling visibility/editability of specific fields like `branch`).
- **Evaluation Baseline:** Each test case explicitly validates two aspects:
    1. **UI Feedback:** Presence or absence of buttons (*New*, *Delete*), UI policies, and form fields.
    2. **Data Level Security:** Visual system messages (e.g., *"Number of rows removed by Security Constraints"*) ensuring hidden records are truly secure, not just visually hidden.

2. Read Access Testing

The EEE User is impersonated and the Student Records list is opened. The table name `u_institution_details.list` can be used to access the list.

The documented verification states that a user with the `bb1` role can view only EEE branch records. This verifies the intended field-based Read access scenario.

A user without the required role is also checked. The documented expected result is that such a user cannot view the records. The Administrator is then tested and is expected to view all records regardless of branch.

- **Role Alignment (`bb1`):** A user assigned only `bb1` can successfully open `u_institution_details.list`, but query-level security filters out non-EEE branch data dynamically.
- **Unauthorized Access Handling:** Users missing the `bb1` role attempting direct URL access (`u_institution_details.list`) encounter a security message and an empty list view with 0 accessible rows.
- **Admin Override Behavior:** System Administrators bypass custom record-level constraints (due to `admin_overrides = true` setting on standard ACLs), confirming default system fallbacks remain intact.
- **Direct Form Query Validation:** Accessing a non-EEE record directly via sys_id URL (e.g., `u_institution_details.do?sys_id=...`) fails with "Record not found" or "Access Denied" alerts.

3. Create and Write Access Testing

Create testing verifies the behavior of the New button and record creation capability. The Create ACL requires bb2.

The documented verification states that a user with bb1 and bb2 roles can view the permitted EEE branch records and use the New button.

Write testing is performed after bb3 is available. The documented verification states that a user with bb1, bb2, and bb3 roles can view and edit the EEE branch records along with the New Button.

- **Create Privilege (bb1 + bb2):**
    - The **New** button becomes visible on the list view header.
    - Form defaults automatically restrict/pre-fill mandatory branch parameters to prevent unauthorized cross-department record creation.
- **Write Privilege (bb1 + bb2 + bb3):**
    - Form fields transition from *Read-only* to *Editable* state upon loading an existing EEE branch record.
    - The **Update** button is rendered active in the form header and context menu.
- **Negative Write Checks:** Users missing bb3 can submit *New* records (bb2), but once saved, the fields immediately lock into read-only mode upon reload.

4. Delete Access Testing and Test Result

Delete testing is performed after bb4 is available. The Delete ACL requires the bb4 custom role.

The documented verification states that users with bb1, bb2, bb3, and bb4 can see, create, edit, and delete the EEE branch records.

The testing phase therefore verifies the progressive operation-level permissions and confirms whether the configured ACLs provide the intended record-level security behavior.

- **Delete Action Verification (bb1 + bb2 + bb3 + bb4):**
    - The **Delete** button appears on the individual record form header.
    - List view row selection checkboxes and the bulk **Delete** choice in *Actions on selected rows* become active.
- **Confirmation Prompt:** Triggering deletion prompts standard system confirmation, ensuring records are removed cleanly without triggering cascade constraints or script errors.
- **RBAC Matrix Summary (Role-Based Access Control):**

Role Combination	Read (EEE Only)	Create (New Button)	Write (Edit Fields)	Delete
No Role	✖	✖	✖	✖
bb1	✔	✖	✖	✖
bb1 + bb2	✔	✔	✖	✖
bb1 + bb2 + bb3	✔	✔	✔	✖
bb1 + bb2 + bb3 + bb4	✔	✔	✔	✔
Admin	✔ (All Data)	✔	✔	✔

- **Final Result Conclusion:** The incremental role execution successfully isolates data by department (EEE) and strictly enforces the Principle of Least Privilege across CRUD operations.

Phase 7 : Project Documentation Phase

1. Project Documentation Overview

- This phase records the complete implementation of the **Script-Controlled ACL** project in a structured form. The documentation explains the purpose of the project, the data model, the roles, the ACL configuration, the script logic, and the verification results.
- The project demonstrates how a script-controlled Access Control List can restrict users from viewing records unless they meet specific conditions.
- The protected table is **`u_institution_details`** and the central field used in the Read ACL is **Branch** (`u_branch`).

2. User, Role, and Table Documentation

- The documented test user is **EEE User**. The project creates custom roles **`bb1`**, **`bb2`**, **`bb3`**, and **`bb4`** and uses them for different access operations.
- The **Institution Details** table uses the system name **`u_institution_details`**. Its fields include:
    - **Student Roll Number:** System Auto Number
    - **Student Name & Faculty Name:** User reference fields (`sys_user`)
    - **Branch:** Choice field containing **ECE**, **EEE**, and **CSE**
    - **Email & Phone Number:** Standard String fields
    - **Description:** Multi-Line String field
- Multiple records are created so that the Read ACL can evaluate records containing different branch values.

3. ACL and Script Documentation

- The **Read ACL** is a record-level ACL for the read operation. It is active, advanced, requires the **`bb1`** role, and contains the **`Branch is EEE`** data condition. Its script checks the administrator role, checks the **`bb1`** role, and denies access when the required logic is not satisfied.
- The **Create ACL** is configured for the create operation and requires **`bb2`**.
- The **Write ACL** is configured for the write operation and requires **`bb3`**.
- The **Delete ACL** is configured for the delete operation and requires **`bb4`**.
- The documentation preserves the configured ACL names, operations, roles, conditions, and verification results so that another person can understand and reproduce the project.

Project Configuration & Implementation Proof

> 📷 **[INSERT YOUR DOCUMENTATION SCREENSHOT HERE]**
>
> *(Recommendation: Add a screenshot showing the list of created ACLs for `u_institution_details` or the active Read ACL script definition)*

4. Final Project Outcome

- By completing the project, learners gain an understanding of how script-controlled **READ**, **WRITE**, **CREATE**, and **DELETE** ACLs enforce record-level security.

- The project demonstrates how user roles and record field values can be evaluated together to make access decisions.
- The ACL configuration protects sensitive data from unauthorized viewing, modification, creation, and deletion.
- The documented outcome covers access behavior across forms, lists, and Service Portal interfaces, providing a complete record-level security demonstration.

Phase 8 : Project Demonstration Phase

1. Demonstration Introduction

- The final demonstration presents the completed **Script-Controlled ACL** project in the **ServiceNow** environment. The demonstration explains how users, roles, records, and ACLs work together to control access.
- Before starting the demonstration, the **EEE User**, custom roles (`bb1`, `bb2`, `bb3`, `bb4`), `u_institution_details` table, required fields, and sample records with **ECE**, **EEE**, and **CSE** values should be available.
- The demonstration focuses on the **Read**, **Create**, **Write**, and **Delete** ACLs and the corresponding access behavior described in the project.

2. Read ACL Demonstration

- The demonstration begins by impersonating the **EEE User** and opening the Student Records list using the path `u_institution_details.list`.
- The configured **Read ACL** is shown with the `bb1` role requirement and the `Branch is EEE` data condition. The advanced script is also presented as the script-controlled part of the access decision.
- The **Administrator** is then impersonated to demonstrate the documented administrative access. The project verification states that the **Administrator** can view all records regardless of branch.

3. Create, Write, and Delete Demonstration

- **Create Access:** Demonstrated using the **New** button after the required **bb2** role is available. The documented verification connects **bb1** and **bb2** with viewing the permitted **EEE** records and accessing the **New** button.
- **Write Access:** Demonstrated after **bb3** is available. The documented verification states that users with **bb1**, **bb2**, and **bb3** can view and edit the **EEE** branch records along with using the **New** button.
- **Delete Access:** Demonstrated after **bb4** is available. The documented verification states that users with all four roles (**bb1**, **bb2**, **bb3**, **bb4**) can see, create, edit, and delete the **EEE** branch records.

Project Demonstration & Execution Proof

> 📷 **[INSERT YOUR DEMONSTRATION SCREENSHOT HERE]**
>
> *(Recommendation: Add a screenshot showing the impersonated EEE User viewing only EEE branch records on the `u_institution_details` list view with active Create/Delete controls)*

4. Final Demonstration Outcome

- The final demonstration shows how the four ACLs create an operation-based security model:
  - **bb1:** Read ACL
  - **bb2:** Create ACL
  - **bb3:** Write ACL
  - **bb4:** Delete ACL
- The project demonstrates that access decisions can be controlled using roles, record field values, data conditions, and script logic. The demonstration provides a practical view of record-level security in ServiceNow.
- The completed project presents a structured security solution in which unauthorized viewing, creation, modification, and deletion are restricted according to the configured ACL rules.